

Lezione 15

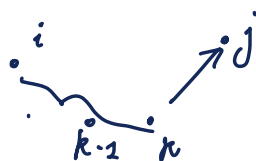
$l_{ij}^{(z)}$: peso di un cammino minimo da i a j contenente al più z archi.

$$l_{ij}^{(0)} = \begin{cases} 0 & i=j \\ \infty & i \neq j \end{cases} \quad z \geq 1, \quad l_{ij}^{(z)} = \min_{1 \leq k \leq n} \{ l_{ik}^{(z-1)} + w_{kj} \}$$

$$L^{(z)} = (l_{ij}^{(z)}) \quad \forall 1 \leq i, j \leq n$$

$\downarrow$

$$L^{(n-1)}$$



$$l_{ij}^{(n-1)} = l_{ij}^{(n)} = l_{ij}^{(n+1)} = \dots = \delta(i, j).$$

15.1 ALGORITMO PER ALL-PAIRS-SHORTEST-PATHS BASATO SULLA PROGRAMMAZIONE DINAMICA E IL PRODOTTO TRA MATRICI

Prese in input la matrice $W = (w_{ij})$, calcoliamo una serie di matrici $L^{(0)}, L^{(1)}, \dots, L^{(n-1)}$ dove $L^{(z)} = (l_{ij}^{(z)})$ per $z = 0, \dots, n-1$.

La matrice finale $L^{(n-1)}$ contiene i pesi dei cammini minimi

$$L^{(n-1)} = (l_{ij}^{(n-1)}) = (\delta(i, j))$$

e pertanto sarà una soluzione al problema.

OSS:
$$l_{ij}^{(z)} = \min_{1 \leq k \leq n} l_{ik}^{(z-1)} + w_{kj}$$

$$C = A \cdot B, \quad A, B, C \text{ } n \times n$$
$$c_{ij} = \sum_{k=1}^n a_{ik} \cdot b_{kj}$$

Esiste una relazione tra il calcolo di $l_{ij}^{(r)}$ e la moltiplicazione tra matrici:

$$\begin{array}{lcl} l_{ij}^{(r)} & \longrightarrow & c_{ij} \\ l_{ik}^{(r-1)} & \longrightarrow & a_{ik} \\ w_{kj} & \longrightarrow & b_{kj} \end{array} \quad \begin{array}{l} + \longrightarrow \cdot \\ \min \longrightarrow + \end{array}$$

Il cuore dell'algoritmo è la procedura EXTEND-SHORTEST-PATHS che estende di un arco i cammini minimi trovati all'iterazione precedente. Calcola $L^{(r)}$ conoscendo $L^{(r-1)}$, W , n

EXTEND-SHORTEST-PATHS ($L^{(r-1)}$, W , $L^{(r)}$, n)

1. for $i=1$ to n
2. for $j=1$ to n
3. for $k=1$ to n
4. $l_{ij}^{(r)} = \min \{ l_{ik}^{(r-1)} + w_{kj}, l_{ij}^{(r-1)} \}$

inizialmente
 $L^{(r)} = (\infty)$

Quindi possiamo risolvere ALL-PAIRS-SHORTEST-PATHS "moltiplicando" ripetutamente delle matrici. Partendo da $L^{(0)}$, $l_{ij}^{(0)} = \begin{cases} 0 & i=j \\ \infty & i \neq j \end{cases}$ e otteniamo

$$L^{(1)} = L^{(0)} \cdot W = W$$

$$L^{(2)} = L^{(1)} \cdot W = W^2$$

$$L^{(3)} = L^{(2)} \cdot W = W^3$$

$\vdots$

$$L^{(n-1)} = L^{(n-2)} \cdot W = W^{n-1}$$

L'algoritmo Slow-APSP calcola queste sequenze di matrici in tempo $\Theta(V^4)$.

SLOW-APSP ($W, L^{(0)}, n$)

1. $L := (l_{ij})$, $M := (m_{ij})$ // due nuove matrici $n \times n$
2. $L = L^{(0)}$
3. for $i=1$ to $n-1$
4. $M = (\infty)$
5. EXTEND-SHORTEST-PATHS(L, W, M, n) // calcolo $M = L \cdot W$
6. $L = M$
7. return L

Q: sono $n-1$ invocazioni di EXTEND-SHORTEST-PATHS che impiegano $\Theta(n^3) = \Theta(V^3)$ ognuna e queste determinano la complessione, pertanto il tempo computazionale globale è $\Theta(n^4)$.

Posiamo diminuire il tempo computazionale?

Osserviamo che a noi interessa $L^{(n-1)}$ e non tutte le $L^{(r)}$ per $0 \leq r \leq n-1$.

Possiamo calcolare $L^{(n-1)}$ eseguendo soltanto $\lceil \log(n-1) \rceil$
prodotti fra matrici usando la tecnica dei quadrati
ripetuti: $L^{(1)} = W$, $L^{(2)} = W \cdot W$, $L^{(4)} = W^2 \cdot W^2 = W^4$
 $\quad\quad\quad \parallel$
 $\quad\quad\quad W^2$

$$L^{(8)} = W^4 \cdot W^4 = W^8, \dots, L^{2^{\lceil \log(n-1) \rceil}} = W^{2^{\lceil \log(n-1) \rceil}} = W^{2^{\lceil \log(n-1) \rceil} - 1} \cdot W^{2^{\lceil \log(n-1) \rceil} - 1}$$

Poiché $2^{\lceil \log(n-1) \rceil} \geq n-1 \Rightarrow L^{2^{\lceil \log(n-1) \rceil}} = L^{(n-1)}$.

L'algoritmo FASTER-APSP implementa queste idee.

FASTER-APSP (W, n)

1. L, M nuove matrici $n \times n$
2. $L = W$
3. $z = 1$
4. while $z < n-1$
5. $M = (\infty)$
6. EXTEND-SHORTEST-PATHS (L, L, M, n)
7. $z = 2z$
8. $L = M$
9. return L

Ognuno dei $\lceil \log(n-1) \rceil$ prodotti impiega $O(n^3)$ -TIME, quindi globalmente il tempo computazionale è $\Theta(n^3 \log n)$.

come prima, il codice non contiene strutture dati elaborate quindi la costante nascosta nella notazione asintotica è piccola.

15.2 L'algoritmo di Floyd-Warshall

L'algoritmo di Floyd-Warshall risolve All-Pairs-Shortest-Paths (a tempo $\Theta(V^3)$).

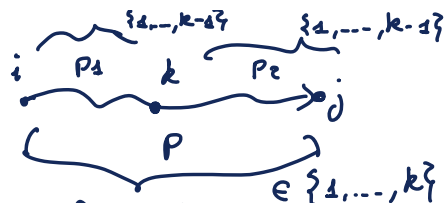
DEF. Un vertice intermedio di un cammino semplice $p = \langle v_s, v_2, \dots, v_e \rangle$ è un qualunque vertice di p diverso da v_s e da v_e , ovvero esso è un vertice appartenente a $\{v_2, \dots, v_{e-1}\}$.

Enumeriamo i vertici di G , $V = \{1, \dots, n\}$ e consideriamo un sottoinsieme $\{1, 2, \dots, k\}$ di vertici per qualche $1 \leq k \leq n$.

OSS: Per ogni coppia di vertici (i, j) consideriamo tutti i cammini da i a j i cui vertici intermedi appartengono a $\{1, \dots, k\}$ e sia p un cammino di per cui valiamo tre questi (p è un cammino semplice).

- se k non è un vertice intermedio di p , allora tutti i vertici intermedi di p appartengono a $\{1, \dots, k-1\}$;
- altrimenti, se k è un vertice intermedio di p , allora possiamo decomporre p in $i \xrightarrow{p_1} k \xrightarrow{p_2} j$, e k non è un vertice intermedio né di p_1 , né di p_2 e quindi i vertici intermedi di p_1 e di p_2 appartengono a

$\{1, \dots, k-1\}$.



Questa osservazione suggerisce una formulazione ricorsiva di ALL-PAIRS-SHORTEST-PATHS diversa da quella vista in precedenza.

Sia $d_{ij}^{(k)}$ il peso di un cammino minimo da i a j i cui vertici intermedi appartengono a $\{1, \dots, k\}$.

$$d_{ij}^{(k)} = \begin{cases} w_{ij} & \text{se } k=0 \\ \min \{ d_{ij}^{(k-1)}, d_{ik}^{(k-1)} + d_{kj}^{(k-1)} \} & k \geq 1 \end{cases}$$

La matrice $D^{(n)} = (d_{ij}^{(n)})$ dà la soluzione al problema, dato che i vertici di un cammino minimo qualunque appartengono a $\{1, \dots, n\}$ quindi $d_{ij}^{(n)} = \delta(i, j) \quad \forall i, j \in V$.

FLOYD-WARSHALL è un algoritmo bottom-up che calcola i valori $d_{ij}^{(k)}$ quindi le matrici $D^{(k)}$ in ordine di k crescente.

FLOYD - WARSHALL (W, n)

1. $D^{(0)} = W$
2. for $k = 1$ to n
3. $D^{(k)} = (d_{ij}^{(k)})$ (move matrix)
4. for $i = 1$ to n
5. for $j = 1$ to n
6. $d_{ij}^{(k)} = \min \{ d_{ij}^{(k-1)}, d_{ik}^{(k-1)} + d_{kj}^{(k-1)} \}$
7. return $D^{(n)}$

Il tempo computazionale di Floyd-Warshall è determinato dal loop annesso (righe 2-6), ogni esecuzione della linea 6 prende tempo $O(1)$, quindi l'algoritmo impiega $\Theta(n^3)$ di moves la costante nascosta nella notazione asintotica è piccola data l'assenza di strutture dati complesse (bound superiore e inferiore).

15.2.1 Costruire uno shortest path

Esistono vari modi per costruire un cammino minimo usando Floyd-Warshall. Uno è calcolare la matrice dei predecessori Π e poi usare la procedura PRINT-ALL-PAIRS-SHORTEST-PATHS.

Alternativamente, la matrice Π può essere calcolata nel

corso dell'algoritmo calcolando $\Pi^{(0)}, \Pi^{(1)}, \dots, \Pi^{(n)}$
 contestualmente a $D^{(0)}, D^{(1)}, \dots, D^{(n)}$.

$\Pi = \Pi^{(n)}$ e $\pi_{ij}^{(k)}$ è definito come il predecessore di j
 in un cammino minimo da i a j i cui vertici
 intermedi sono in $\{1, \dots, k\}$.

Formalmente,

$$\pi_{ij}^{(0)} = \begin{cases} NIL & \text{se } i=j \text{ o } w_{ij} = \infty \\ i & \text{se } i \neq j \text{ e } w_{ij} < \infty \end{cases}$$

$$k > 0 \quad \pi_{ij}^{(k)} = \begin{cases} \pi_{kj}^{(k-1)} & \text{se } d_{ij}^{(k-1)} > d_{ik}^{(k-1)} + d_{kj}^{(k-1)} \quad \begin{array}{l} k \text{ \u00e9 un} \\ \text{vertice} \\ \text{intermedio} \end{array} \\ \pi_{ij}^{(k-1)} & \text{se } d_{ij}^{(k-1)} \leq d_{ik}^{(k-1)} + d_{kj}^{(k-1)} \quad \begin{array}{l} k \text{ non} \\ \text{\u00e9 un} \\ \text{vertice} \\ \text{intermedio} \end{array} \end{cases}$$